REPORT PROJECT 1 (AyamGepuk).pdf

by Muhammad Naim abdullah

Submission date: 25-May-2026 07:43AM (UTC-0700)
Submission ID: 2969198878
File name: REPORT_PROJECT_1_AyamGepuk_.pdf (1.1M)
Word count: 2946
Character count: 15868

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 - 01

HIGH PERFORMANCE DATA PROCESSING

PROJECT 1

LECTURER**: DR MOHD SHAHIZAN BIN OTHMAN**

GROUP **: AyamGepuk**

	NAME	**MATRIC NO**
1.	MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
2.	MUHAMMAD NAIM BIN ABDULLAH	A23CS0134
3.	SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183

Table of Contents

1.0 Introduction

1.1 Background of the Project

Online news platforms have been an important source of information on various topics including business, finance, politics, social issues and events happening all around us. As public data increases on the internet, especially news websites, web scraping has emerged as a powerful tool to automate the collection of large amounts of contents and information in order to conduct sentiment analysis, trend forecasting and content archiving.

This project uses web scraping techniques to extract news articles from two established Malaysian news portals which are Edge Malaysia and The Sun by deploying a high-performance asynchronous scraping pipeline. The end goal is to gather at least 100,000 news articles records for further data processing and performance benchmarking and evaluation.

1.2 Objectives

- To develop a web scraper capable of acquiring over 100,000 news articles records from two target websites
- To extract specific attributes such as article ID, title, author, source, summary, link and publication date.
- To store the extracted data in a structured CSV format for processing and performance benchmarking
- To clean and preprocess the merged dataset to ensure data quality
- To compare three data processing libraries, Pandas, Dask and Polars across multiple performance metrics
- To optimize the data processing pipeline and evaluate performance improvements

1.3 Target Website and Data

The websites targeted for this web scraping project are:

1. Edge Malaysia (https://theedgemalaysia.com), a leading Malaysian financial and business news portal, scraped via its internal category API.
2. The Sun (https://thesun.my), a Malaysian daily newspaper, scraped via its WordPress REST API.

The specific data fields extracted from each article are:

- article_id: A unique identifier prefixed with EDGE_ or SUN_ depending on source.
- category: The main topic area of the article.
- sub-category: A secondary classification label.
- title: The headline of the article.
- author: The article's author or reporter.
- source: The originating publication.
- summary: A short excerpt or description of the article content.
- link: The full URL of the article.
- created date: The publication date and time (standardized to D/M/YYYY H:MM:SS AM/PM).
- updated date: The last modified date and time.

2.0 System Design & Architecture

2.1 Description of architecture

This system architecture shows the workflow of the web scrapping and data preprocessing pipeline that extract, clean and store the news data from the Edge Malaysia and The Sun. This system follows the data flow architecture pattern where data move in sequence of orders.

1. API
   The system begins with requesting the Edge Malaysia category API and The Sun Wordpress API which will be used as the seed inputs.

2. Asynchronous Web Crawler
   Both scrapers are using asyncio and aiohttp for asynchronous HTTP requests. asyncio.semaphore is being used to control the concurrency by setting The Edge for 50 and 10 for The Sun to avoid overloading servers.

3. Raw Data Storage
   The Edge data is written in append-mode batches directly to theedge.csv to manage memory during large-scale collection. The Sun data is collected in memory and saved to thesun.csv after deduplication.

4. Data Merging & Cleaning
   pd.concat is being used to merge both raw csv, deduplicated on the link column and cleaned before being saved as FINAL_MASTER_100K.csv.

5. Optimization & Performance Benchmarking
   The ETL pipeline is used in the pandas, dask and polars. Each of this libraries will be evaluated based on their execution time, memory usage, CPU utilization and throughput.

6. Data Storage

   The final cleaned and benchmarked dataset is saved in CSV format.

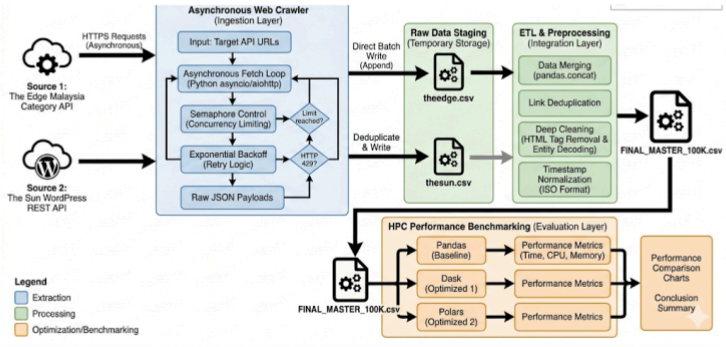
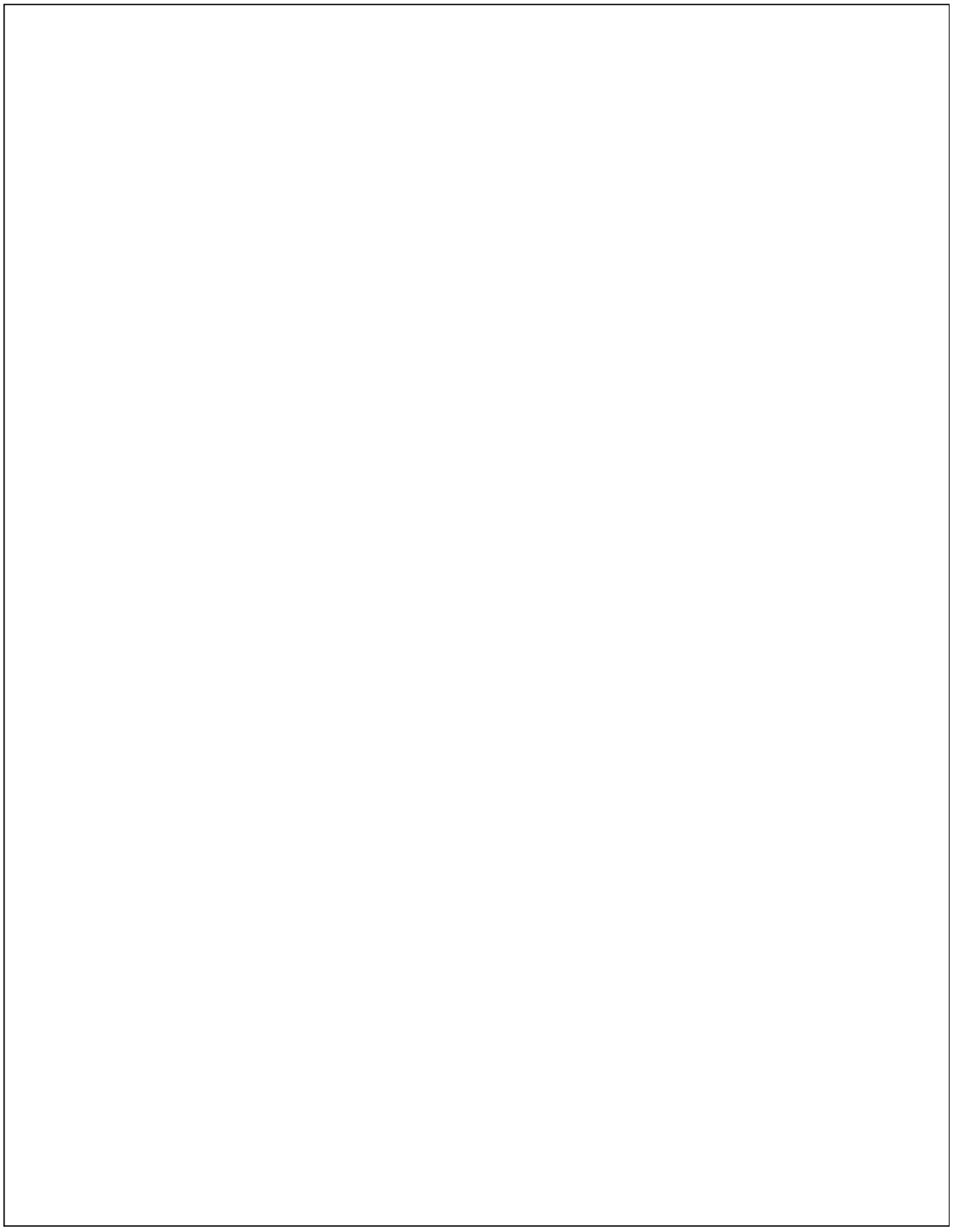


Figure 2.1 Architecture design of the web scrapping and data preprocessing pipeline

2.2 Tools and Frameworks Used

The pipeline utilizes these tools and framework to achieve high performance and accuracy.

Tools and framework	Functional
asyncio & aiohttp	For asynchronous web extraction.
BeautifulSoup	For HTML parsing
Pandas, Dask, Polars	For data processing and performance benchmarking
psutil	For hardware resource monitoring
Matplotlib	For data visualization



2.3 Roles of the Team Members

To ensure a balanced workload and efficient project execution, the team responsibilities were distributed as follows:

Team Member	Designated Role	Key Responsibilities
MUHAMMAD NAIM BIN ABDULLAH	Group Leader and Coder	Oversees project milestones and coordinates team efforts. Leads the optimization phase by implementing High-Performance Computing (HPC) techniques using Dask and Polars. Responsible for conducting system benchmarks and generating execution performance charts.
MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	Architect and coder	Designs the overall data flow architecture. Develops the highly concurrent asynchronous web scraping pipeline utilizing asyncio and aiohttp. Manages pagination, rate-limiting (exponential backoff), and the initial raw data ingestion.
SYARIFAH DANIA BINTI SYED ABU BAKAR	Analyst, Documentation Lead and Coder	Drives the data preprocessing phase, including HTML

		stripping, deduplication, and missing value imputation. Manages the final data formatting (CSV) and leads the compilation, structuring, and refinement of the final project report and documentation.

3.0 Data Collection

3.1 Crawling Method

Data collection was performed using two scrapers implemented with asyncio and aiohttp.

- Edge Malaysia Scraper targets the internal REST API endpoint, scraping 44 categories like corporate, economy and others. This script run up to 50 parallel requests using an asyncio semaphore. It also saves data to csv file in batches of 100 to keep RAM usage low
- The Sun targets the WordPress REST API, fetching 150 pages at 50 records per page. It uses up to 10 concurrent requests to respect the server's stricter limits.
- Timestamps are normalized at extraction time which is UNIX milliseconds for The Edge and ISO strings for The Sun are converted to D/M/YYYY H:MM:SS AM/PM. HTML tags are stripped and entities decoded using re.sub and html.unescape() during extraction.

3.2 Number of Record Collected

The objective of collecting 100,000 records was successfully achieved. The breakdown is as follows:

- Edge Malaysia: 93,249 unique records
- The Sun: 7,500 unique records
- Final Master Dataset (after deduplication): 100,749 unique records

Edge Malaysia was scraped across 44 categories, and the total scraping time for The Edge was highly optimized, completing in approximately 365 seconds.

3.3 Ethical Considerations

The web scrapers were developed with strict adherence to ethical guidelines to ensure responsible data collection:

- **Respect site rules:** Only publicly accessible API endpoints and pages were accessed, ensuring scraping did not violate the websites' terms of service.
- **Rate limiting:** Concurrency limits (Semaphores) and exponential backoff mimic responsible API usage, significantly reducing the risk of server overload or IP bans.
- **Non-commercial use:** The collected data is strictly for academic and research purposes, with no intention of redistributing the scraped content.

4.0 Data Processing

4.1 Cleaning Methods

4.1.1 HTML Cleaning (Applied during Extraction)

Raw API responses involve HTML markup. A clean_text() function is applied immediately during scraping as shown in Figure 4.1.

```
def clean_text(text):
    if not isinstance(text, str): return ""
    text = re.sub(r'<.*?>', '', text)   # Remove HTML tags
    text = html.unescape(text)          # Decode &amp;, &quot; etc.
    return re.sub(r'\s+', ' ', text).strip()
```

Figure 4.1 Applying clean_text() function

4.1.2 Handling duplicated data

Articles can appear under multiple different categories or in overlapping API results. Duplicates are removed based on the unique article link as depicted in Figure 4.2.

```
master_df.drop_duplicates(subset=['link'], keep='first', inplace=True)
```

Figure 4.2 Query to remove duplicate data

4.1.3 Handling missing data

Empty cells are filled with "N/A" to produce a clean, production-ready dataset, shown in Figure 4.3 below.

```
master_df.fillna("N/A", inplace=True)
```

Figure 4.3 Query to reformat missing data

4.2 Data Structure

Raw data from each source is saved independently as edge_df and sun_df,, both sources are merged using Pandas as shown in Figure 4.4.

```
master_df = pd.concat([edge_df, sun_df], ignore_index=True)
```

Figure 4.4 Merging article records from The Edge and The Sun.

4.3 Transforming and Formatting

4.3.1 Timestamp Normalization

The Edge provides UNIX millisecond timestamps; The Sun provides ISO 8601 strings. Both are converted to a consistent format (D/M/YYYY H:MM:SS AM/PM) as displayed in Figure 4.5.

4.3.2 Unique Article ID

Each record is assigned a prefixed unique ID at extraction time to distinguish source after merging as shown in Figure 4.6 below.

```
# The Edge
"article_id": f"EDGE_{item.get('nid', '00000')}"

# The Sun
"article_id": f"SUN_{post.get('id', '00000')}"
```

Figure 4.5 Assigning unique ID for each article

5.0 Optimization Techniques

5.1 Method Used

In this project, there are 3 data processing libraries that been used to do the same 5 task ETL pipeline to 100,000 row master dataset :

Library	Explanation
Pandas	The standard python data library. It uses eager execution where loads the entire dataset into RAM at once and processes operation one after another on a single thread. Used as the baseline.
Dask	A parallel computing library that scales Pandas workflows. Constructs a directed acyclic graph (DAG) of deferred tasks and executes them in parallel across multiple CPU cores via its task scheduler. Designed for out-of-core (larger-than-RAM) datasets.
Polars	A modern High performance Dataframe library built in Rust. Uses a lazy evaluation with query optimization, and uses all CPU cores at the same time without being stopped by Python's GIL

5.2 Benchmark Pipeline

The same pipeline is implemented identically in each library :

Task 1 : Data Ingestion

- Load the master CSV from disk

```
# Pandas
df = pd.read_csv(FILE_PATH)

# Dask
ddf = dd.read_csv(FILE_PATH)

# Polars (Lazy)
q = pl.scan_csv(FILE_PATH)
```

Figure 5.1 Query Data Ingestion

Task 2 : Feature Engineering

- Create a new title_length column based on string length

```
# Pandas / Dask
df['title_length'] = df['title'].str.len()

# Polars
.with_columns([pl.col("title").str.len_chars().alias("title_length")])
```

Figure 5.2 Query create new column

Task 3 : Data Standardization

- Normalize the category column to uppercase

```
# Pandas / Dask
df['category'] = df['category'].str.upper()

# Polars
.with_columns([pl.col("category").str.to_uppercase()])
```

Figure 5.3 Query Data Standardization

Task 4 : Type Casting and Date Parsing

- Convert the created date string column to datetime objects

```
# Pandas
df['created date'] = pd.to_datetime(df['created date'], format='%d/%m/%Y %I:%M:%S %p', errors='coerce')

# Dask
ddf['created date'] = dd.to_datetime(ddf['created date'], format='%d/%m/%Y %I:%M:%S %p', errors='coerce')

# Polars
.with_columns([pl.col("created date").str.to_datetime("%d/%m/%Y %I:%M:%S %p", strict=False)])
```

Figure 5.4 Query Type Casting and Date Parsing

Task 5 : Multi-Dimensional Aggregation

- Group by source and category, then compute mean title length and article count

```python
# Pandas
result = df.groupby(['source', 'category']).agg({
    'title_length': 'mean', 'article_id': 'count'
}).reset_index()

# Dask (triggers parallel execution)
result = ddf.groupby(['source', 'category']).agg({
    'title_length': 'mean', 'article_id': 'count'
}).compute()

# Polars
.group_by(["source", "category"]).agg([
    pl.col("title_length").mean().alias("avg_len"),
    pl.len().alias("count")
])
result = q.collect()
```

Figure 5.5 Query Multi-Dimensional Aggregation

6.0 Performance Evaluation

The baseline pipeline (Pandas) operates on a single thread with eager execution. The same pipeline was re-implemented in Dask and Polars to measure improvement. Each library was run 3 times on the 100,000-row master dataset, with memory cleared between runs (gc.collect()) to ensure isolation. Performance metrics were averaged across all runs.

6.1 Detailed Result Per Run

Library	Run	Execution Time (s)	Memory Used (MB)	CPU Util (%)	Throughput (rows/s)
Pandas	1	1.6060	42.93	99.62	62,265.55
Pandas	2	1.8461	30.43	97.50	54,167.80
Pandas	3	1.5362	28.46	100.25	65,094.36
Dask	1	1.3147	12.56	100.40	76,063.58
Dask	2	1.3303	0.00	100.73	75,168.35
Dask	3	1.7315	9.88	98.18	57,751.97
Polars	1	0.1104	3.77	154.04	906,101.38
Polars	2	0.0942	0.25	191.12	1,061,776.64
Polars	3	0.0916	1.66	196.43	1,091,275.95

Figure 6.1 Result Per Run

6.2 Average Summary

Library	Execution Time (s)	Memory Used (MB)	CPU Util (%)	Throughput (rows/s)
Pandas	1.6628	33.94	99.12	60,509
Dask	1.4588	7.48	99.77	69,661
Polars	**0.0987**	**1.89**	**180.53**	**1,019,718**

Figure 6.2 Result Average Summary

6.3 Charts and Graphs

To visually summarize the differences between Pandas, Dask, and Polars performance, a series of bar charts were created based on the average performance result.

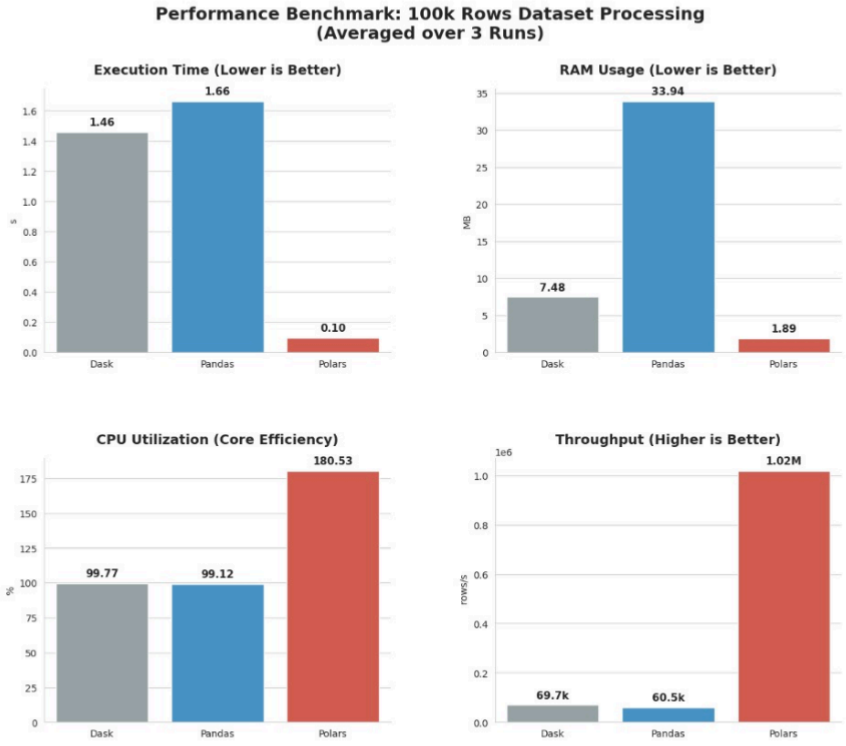


Figure 6.3 Comparison Performance Chart

No	Title	Description
1.	Comparison of Execution Time	Polars completed the pipeline in under 0.1 seconds on average where it was 17 times faster than Pandas and 15 times faster than Dask. Dask recorded a modest improvement over Pandas.
2.	Comparison of Memory Usage	Polars used the least memory, attributed to its lazy evaluation engine and projection pushdown which avoids loading unnecessary data into RAM. Dask also performed well by processing data in partitions. Pandas consumes the most RAM as it loads the entire dataset eagerly.
3.	Comparison of CPU Utilization	Polars achieved CPU utilization exceeding 180%, demonstrating effective use of multi core CPU via its Rust engine ignoring the Python's Global Interpreter Lock (GIL). Pandas and Dask both stayed near 100%, indicating single core or lightly parallel execution.
4.	Comparison of Throughput	Polars processed over 1 million rows per second, compared to 69,661 rows/s for Dask and 60,509 rows/s for Pandas

6.4 Conclusion

Based on the result, Polars is the best library to use because it outperforms the Pandas and Dask in every category. Polars runs 17 times faster than Pandas, processes up to 1 million rows per second, and uses the amount of RAM by loading the necessary data only. Furthermore, Polars successfully breaks free from Python’s speed limits to use multiple cores at the same time, making it the most efficient for data processing.

7.0 Challenges & Limitations

Building this 100,000-row dataset came with several real-world engineering hurdles. Scraping *The Edge Malaysia* and *The Sun* required navigating strict anti-bot protections and managing tight memory limits within Google Colab. Additionally, reconciling inconsistent data formats and dealing with the unexpected system overhead of Dask added extra complexity to the pipeline.

The specific challenges and the workarounds used to solve them are detailed below:

1. Anti-Scraping Measures: Both Edge Malaysia and The Sun employ server-side protections. HTTP 429 errors were encountered during The Edge scraping. This was mitigated with exponential backoff (asyncio.sleep(2 ** attempt)) and concurrency caps, but such measures can limit data collection speed.
2. Scale and Memory Management: Collecting 93,249 records from The Edge required writing data in append-mode batches directly to CSV rather than holding all records in RAM, since loading hundreds of thousands of rows in memory simultaneously is infeasible in a Colab environment.
3. Data Quality and Inconsistency: API responses from the two sources had different structures, timestamp formats (UNIX milliseconds vs ISO strings), and HTML-encoded content. Normalizing these at extraction time required separate transformation logic per source.
4. Merging Data from Multiple Sources: Aligning schemas between Edge Malaysia and The Sun required careful mapping. Since both sources were independently scraped, post-merge deduplication on the link column was necessary to ensure exactly 100,000 distinct rows in the final dataset.
5. Dask Scheduler Overhead on Medium Data: For a 100,000-row dataset, Dask's task graph construction and scheduler overhead reduced its advantage over Pandas. Dask's strength is better realized on datasets that exceed available RAM.

8.0 Conclusion & Future Work

8.1 Summary and Findings

This project successfully designed and built a high-performance asynchronous web scraping pipeline using *asyncio* and *aiohttp* to extract over 100,000 news articles records from Edge Malaysia and The Sun. The Edge Malaysia scraper uses its internal category API across 44 categories while The Sun scraper uses the WordPress Rest API which enables a collection of 93,249 and 7,500 records respectively within minutes.

After merging, removing duplicates and data cleaning, the master dataset (FINAL_MASTER_100K.csv) was used as the benchmark input for comparing three data processing libraries. Pandas is set as the single-threaded eager baseline. Dask offere modest improvement through parallel task scheduling with lower memory consumption. Polars delivered remarkable results, processing over 1 million rows per second at under 0.1 seconds, using least possible memory and achieving over 180% CPU utilization through its native Rust multi-threading engine. The results confirmed that choosing the right library fruits a dramatic impact on performance at scale.

8.2 What Could be Improved

1. Save collected data into a database: Currently, data is persisted as CSV files. Storing data in a database such as PostgreSQL or MongoDB would enable more efficient querying, better indexing and scalable storage for growing datasets.

2. Expand The Sun collection: Only 150 pages were fetched from The Sun's API (7,500 records). Pagination can be extended further, or additional category endpoints explored, to gather a larger and more balanced dataset from that source.
3. Use distributed scraping across machines: Deploying scrapers across multiple machines or cloud workers simultaneously would allow collection to scale far beyond what a single Colab session supports, particularly for The Edge's 40+ categories.
4. Improve fault tolerance with checkpointing: If the Colab session times out mid-scrape, all progress is lost. Implementing periodic checkpoint saves (e.g., every 10,000 records) would ensure long-running scrapes are resilient to interruptions.
5. Benchmark on larger datasets: The 100,000-row dataset did not fully expose Dask's strengths. Testing all three libraries on datasets of 1M+ rows or datasets exceeding available RAM would provide a more comprehensive picture of Dask's distributed computing advantage.

9.0 References

Dask. (n.d.). Dask: Scale the Python tools you love. https://docs.dask.org/

IBM. (2024, July 9). HPC. IBM. https://www.ibm.com/think/topics/hpc

NetApp. (n.d.). What is high-performance computing (HPC)? How it works. NetApp. https://www.netapp.com/data-storage/high-performance-computing/what-is-hpc/

Perez, M. (2019, August 6). What is web scraping and what is it used for? ParseHub Blog. https://www.parsehub.com/blog/what-is-web-scraping/

Polars. (n.d.). User guide. https://docs.pola.rs/

Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. Proceedings of the 14th Python in Science Conference (SciPy 2015), 130–136. https://doi.org/10.25080/Majora-7b98e3ed-013

VanderPlas, J. (2016). Python data science handbook: Essential tools for working with data. O'Reilly Media. https://jakevdp.github.io/PythonDataScienceHandbook/

Wikipedia contributors. (2019, October 4). Web scraping. Wikipedia. https://en.wikipedia.org/wiki/Web_scraping

REPORT PROJECT 1 (AyamGepuk).pdf

ORIGINALITY REPORT

13%	2%	0%	12%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Universiti Teknologi Malaysia Student Paper	12%
2	hyno.co Internet Source	<1%
3	www.coursehero.com Internet Source	<1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		